Motion Rush - Project Brief

Motion Rush

A Deep Learning-Powered, Camera-Controlled Endless Runner (Subway Surfers-style)

Prepared as a project brief — final workflow and development plan

---

1. Project Objective

Motion Rush is a full-body, camera-controlled endless runner inspired by Subway Surfers. Instead of touch or keyboard input, the player stands in front of a webcam and controls the character using physical movements — jumping, squatting, and side-stepping left/right. The system captures video in real time, estimates the player's body pose using a deep learning model, classifies the pose sequence into a discrete game action, and translates that action into a game control event (keypress, swipe-equivalent, or in-engine signal).

---

2. End-to-End Project Flow

1. **Camera setup:** Player stands ~1.5–2m from the webcam in a calibration zone. System captures a few seconds of a neutral standing pose to calibrate body proportions and frame boundaries.
2. **Frame capture:** Webcam feed is captured at a fixed rate (e.g. 20–30 FPS) and resized/normalized for the pose model.
3. **Pose estimation:** Each frame (or short window of frames) is passed through a pose-estimation model to extract body keypoints (shoulders, hips, knees, ankles, wrists, head).
4. **Feature extraction:** Convert raw keypoints into motion-relevant features — joint angles, vertical displacement of hips/shoulders, lateral displacement of the torso center, velocity between frames.
5. **Gesture classification:** A temporal model (sequence of frames, not a single frame) classifies the motion into one of a fixed action set: Jump, Squat/Duck, Move-Left, Move-Right, Idle/Run.
6. **Debounce & confirmation:** Reject flickering/low-confidence predictions; require a short confirmation window before an action is confirmed, so a single noisy frame doesn't trigger a false action.
7. **Action mapping:** Confirmed gesture is mapped to a game control event (simulated key press for the actual Subway Surfers app, or a direct input event if using a custom-built runner).
8. **Game engine update:** The runner game (custom-built or automated externally) executes the action — lane switch, jump animation, duck animation, obstacle collision check.
9. **Feedback loop:** On-screen overlay shows the player's detected pose/skeleton and the recognized action, so the player can see why an action did or didn't register.
10. **Session state update:** Track current lane, score, combo/streak, last action, cooldown timers.
11. **Run until game over:** Continue until collision/game-over condition; then show final score, run stats (gesture accuracy, reaction time), and reset session state while keeping the models loaded in memory.

---

3. Development Plan

The project is split into independent modules so pose estimation, gesture classification, and game control can each be built and validated before wiring up the live loop.

Layer / technology	What it does in the project
Camera input — OpenCV	Captures webcam frames, handles resolution/FPS, draws the debug overlay (skeleton, detected action, FPS counter).
Pose estimation — MediaPipe Pose / BlazePose (or MoveNet)	Real-time, lightweight full-body keypoint detection (33 landmarks); runs on CPU at usable FPS, no GPU required for the base version.

Gesture/action recognition — CNN + LSTM or 1D-CNN over keypoint sequences	Takes a sliding window of keypoint feature vectors and classifies the motion as Jump / Squat / Left / Right / Idle. Sequence model is needed because a "jump" is a *motion over time*, not a single-frame pose.
Calibration module — custom Python	Establishes each player's neutral pose, shoulder width (for left/right thresholds), and standing hip height (for jump/squat thresholds) at session start.
Game control bridge — PyAutoGUI / pydirectinput (or custom input API)	Converts a confirmed gesture into a keyboard event (arrow keys) if driving the real Subway Surfers app, or into a direct function call if driving a custom-built runner.
Game engine — Pygame / Godot / HTML5-Canvas (custom runner)	Optional but recommended: a simplified custom endless-runner (3-lane track, obstacles, coins, jump/duck mechanics) so the team fully controls hit-detection and doesn't depend on emulating a third-party mobile game.
UI/Overlay — OpenCV / Tkinter / web front-end	Calibration screen, live skeleton + action-label overlay, score display, end-of-run summary.
Backend/session logic — Python (FastAPI optional)	Session state, calibration parameters, gesture history, cooldown/debounce timers, score tracking.
Data logging — CSV / SQLite	Logs gesture predictions, confidence scores, and timestamps per run — used for accuracy evaluation and threshold tuning.

---

4. Proposed Project Structure

```
motion_rush/
├── camera/            # frame capture, calibration, FPS handling
├── pose/               # pose-estimation wrapper (MediaPipe/MoveNet), keypoint extraction
├── gesture/            # feature engineering, sequence model, training scripts
│   ├── models/         # saved model weights
│   └── data/           # recorded gesture clips for training/testing
├── control/            # gesture-to-action mapping, debounce logic, input bridge
├── game/               # custom endless-runner (if built in-house)
├── ui/                 # calibration screen, live overlay, HUD
├── logs/               # session logs, accuracy/latency records
├── tests/              # unit tests for pose parsing, gesture thresholds, control mapping
└── config/             # thresholds, cooldowns, FPS targets, key bindings
```

---

5. Core DL / CV Architecture

A. Pose Estimation Backbone. Use a pretrained, real-time pose model (MediaPipe Pose or MoveNet Lightning) rather than training a pose estimator from scratch — this is the single biggest scope-saver for the project and still gives 30+ FPS on CPU. Output: 33 (or 17) 2D/3D keypoints per frame with visibility/confidence scores.

B. Feature Engineering. Raw (x, y) keypoints are not directly usable across different body sizes/camera distances. Normalize by shoulder width and hip-center position from calibration. Derive: vertical hip displacement (jump/squat), horizontal shoulder-center displacement (left/right), knee-bend angle (squat), and frame-to-frame velocity of the hip center (distinguishes a fast jump from a slow crouch).

C. Temporal Gesture Classifier. A single frame cannot tell "about to jump" from "standing" — the model needs a short window (e.g. 10–15 frames, ~0.5s) of feature vectors. Two viable architectures: - **Lightweight:** a small 1D-CNN or GRU/LSTM over the feature-vector sequence — fast, easy to train on a modest self-recorded dataset. - **Rule-assisted hybrid:** simple threshold rules (hip drops below X% → squat; hip rises above Y% → jump) as a fast fallback, with the learned model used to resolve ambiguous or borderline cases. This hybrid approach is recommended for the exhibition build because it's far more robust with limited training data.

D. Debounce & Cooldown Logic. Not a DL component, but essential: each action has a minimum cooldown (e.g. 300ms) before it can re-trigger, and a confidence threshold + N-consecutive-frame confirmation window prevents a single noisy pose from firing an unintended action.

E. Calibration Layer. Every player has different height, arm length, and standing posture. A 3–5 second calibration phase captures a personalized baseline so jump/squat/left-right thresholds are relative to *that* player, not a fixed global value.

---

6. Real-World Issues and How We Address Them

Issue	Planned handling
Poor/variable lighting	Use a pose model trained to be lighting-robust (MediaPipe); add a brightness/contrast normalization pre-step; recommend a well-lit demo area.
Player too close/far from camera or partially out of frame	Calibration step checks that all key landmarks are visible before allowing gameplay to start; on-screen guide box tells the player to reposition.
Background clutter / other people walking through frame	Track a single primary skeleton (largest/most-central detection); ignore secondary detections; optional simple background subtraction.
False positives (accidental jump/squat while just shifting weight)	Confidence threshold + N-consecutive-frame confirmation + per-action cooldown timer.
False negatives (real jump not detected)	Tune thresholds using logged gesture data; hybrid rule+model approach catches edge cases the model alone might miss.
Left/right confusion or camera mirroring	Explicit mirror-mode flag; use calibrated shoulder-center as the reference axis, not absolute pixel position.
Latency between move and game reaction	Pose inference and action mapping kept in a fast local loop; no network dependency in the control path; target end-to-end latency under ~150ms.
Different body types / heights / clothing	Calibration normalizes thresholds per player rather than using fixed pixel values; loose clothing affects torso landmarks less than limb landmarks, so core logic relies on hip/shoulder/knee points.
Camera resolution / low-end webcams	Downscale frames to a fixed low resolution (e.g. 480p) before pose inference to keep FPS stable on modest hardware.
Player fatigue over long sessions (physical game)	Fixed-length rounds/checkpoints for the exhibition; visible cooldown indicators so players understand action pacing.
Multiple simultaneous actions (e.g. jump while moving side)	Priority ordering (e.g. vertical action overrides lateral in the same window) and a short lockout after each confirmed action.
Third-party game (real Subway Surfers) not accepting automated input reliably	Recommend building a lightweight in-house 3-lane runner clone for the exhibition so the team fully controls the input pipeline and hit-detection, avoiding anti-automation or emulator quirks.

---

7. Gesture Set & Action Mapping

Player movement	Detected via	Mapped game action
Quick upward hip/shoulder motion	Vertical velocity spike + hip displacement above threshold	Jump (avoid low obstacle / gap)
Quick downward crouch	Knee-angle bend + hip drop below threshold, held briefly	Squat/Duck (avoid overhead obstacle)
Lateral step/lean left	Shoulder-center shifts left of calibrated center beyond threshold	Move to left lane
Lateral step/lean right	Shoulder-center shifts right of calibrated center beyond threshold	Move to right lane
Neutral standing/running-in-place	No threshold crossed	Idle/continue current lane

---

8. Example of Gesture-Level Verification

Sequence: player leans left, then squats, then jumps within a 3-second window. - **Segment 1:** classified independently as “Move-Left” — evaluated on its own feature window, confidence logged. - **Segment 2:** classified independently as “Squat” — a new calibration-relative threshold check, separate from Segment 1. - **Segment 3:** classified independently as “Jump” — velocity-based detection, separate cooldown timer from squat.

Each segment is evaluated independently rather than as one blended motion, which prevents one strong, unambiguous gesture (e.g. a clean jump) from masking a weak or ambiguous one (e.g. a partial lean that shouldn’t have triggered a lane change).

---

9. Model Evaluation Metrics

Metric	What it measures
Per-gesture precision/recall	How often each action (jump/squat/left/right) is correctly detected vs. missed or falsely triggered
End-to-end latency	Time from physical movement completion to on-screen game reaction
False-trigger rate	Actions fired with no corresponding intentional player movement
Calibration success rate	% of sessions where calibration completes without repositioning
FPS / inference time	Frames processed per second on target exhibition hardware

These are engineering targets, not fixed guarantees — they should be tuned against a small self-recorded gesture dataset before the exhibition.

---

10. Live Session State

Stored state	Purpose
Calibration parameters	Per-player thresholds for jump, squat, left, right, derived from the neutral pose.
Current lane / vertical state	Whether the player is currently left/center/right lane, standing/jumping/ducking.
Gesture history	Recent confirmed actions, used for cooldown enforcement and debug overlay.
Cooldown timers	Prevents an action from re-firing before its minimum interval has passed.
Score & run stats	Distance/score, coins, obstacles avoided, gesture accuracy for the run.
Frame buffer	Rolling window of recent keypoint features feeding the sequence classifier.

---

11. Latency and Reliability Strategy

- Load the pose-estimation model once at application startup; never reload per session.
- Keep the pose→gesture→control loop fully local — no network calls in the real-time control path.
- Downscale camera frames before inference to hold a stable FPS on exhibition hardware.
- Use a fixed-size rolling buffer for the sequence classifier so inference time doesn’t grow over a long session.
- Debounce and cooldown logic prevents jitter from cascading into rapid, unintended repeated actions.
- Graceful degradation: if pose confidence drops (player briefly out of frame), pause action detection rather than firing on incomplete data, and show a clear on-screen prompt.
- On game-over/reset, clear only per-run state (score, lane, history) — keep the pose model and calibration loaded for

the next player.

---

12. Robustness and Exhibition Test Plan

Test area	What we will measure
Gesture accuracy	Precision/recall per gesture across multiple testers of different heights/builds.
Calibration robustness	Success rate across different lighting conditions and distances from camera.
False-trigger resistance	Rate of accidental actions during normal standing/idle movement.
Latency	Time from physical action to on-screen response, median and worst-case.
Multi-session stability	Repeated calibration → play → reset cycles without crashes or state leakage between players.
Camera/hardware variance	Behavior across different webcam qualities and exhibition lighting.
Player onboarding	How quickly a first-time player understands the controls without instructions.

---

13. Final Build Order

1. **Pose pipeline prototype:** Get MediaPipe/MoveNet running live with an on-screen skeleton overlay and FPS counter.
2. **Feature + threshold prototype:** Build calibration and raw threshold-based detection (rule-based first) for jump/squat/left/right.
3. **Gesture dataset + classifier:** Record sample gesture clips from multiple people; train the lightweight sequence model; compare against the rule-based baseline.
4. **Debounce & control bridge:** Add cooldown/confirmation logic and wire confirmed gestures to keyboard/game input.
5. **Game integration:** Build (or connect to) the endless-runner game loop — lanes, obstacles, collision, scoring.
6. **UI/overlay & calibration screen:** Player-facing calibration flow, live skeleton overlay, score HUD.
7. **Hardening:** Stress-test across multiple players, lighting conditions, and repeated sessions; tune thresholds from logged data.

> **Engineering note:** No pose-based control system will be 100% accurate for every body type and environment. The goal is a responsive, calibration-aware system with sensible fallbacks (rule-based thresholds backing up the learned model), clear visual feedback, and thorough testing across multiple testers before the exhibition.